Tarea 2: Multiplicación de Matrices con OpenMP y Numba

IIC3533 – Computación de Alto Rendimiento

Gonzalo Rivas Valenzuela Diego García Javier Curivil

Octubre 2026

1. Base serial y tamaño de bloque

Ejecutamos 10 repeticiones para cada tamaño de bloque n con $N = 2048$ y tomamos el promedio.[1]

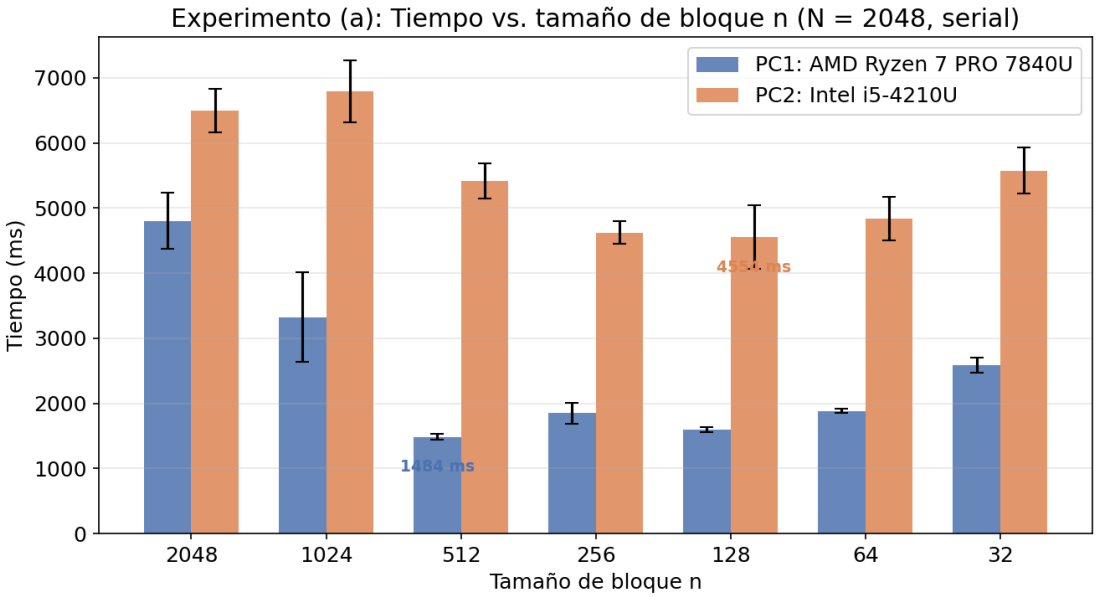


Figura 1: Tiempo versus tamaño de bloque (n).

El parámetro n controla el tamaño de los bloques en los casos base de la recursión; es decir, la matriz original se divide hasta llegar a submatrices de $n \times n$, generando un total de $(N/n)^2$ hojas donde se ejecuta la multiplicación y luego se suman los resultados. Así, el comportamiento de los tiempos forma una curva en U debido a dos factores: para n grande, los bloques no caben en la memoria caché causando muchos *cache misses*; y para n pequeño, la

[1] Compilamos con `g++ -O3 -march=native -fopenmp serial.cpp`, aunque el programa no usa OpenMP.

cantidad de subproblemas y el *overhead* de la subdivisión crecen excesivamente (por ejemplo, con $n = 32$ hay 4096 hojas y 6 niveles de profundidad).

Dado lo anterior, los valores óptimos ($n = 512$ en PC1 y $n = 128$ en PC2) coinciden exactamente con los tamaños de bloque cuyo *working set* cabe razonablemente en la caché L2 de cada procesador.[2]

2. Algoritmo 1: Paralelismo de Datos con OpenMP

2.1. Implementación

Se implementó en C++ el Algoritmo 1 aplicando paralelismo de datos mediante OpenMP sobre el bucle exterior de la multiplicación matricial densa de $N \times N$ ($N = 2048$), utilizando la directiva `#pragma omp parallel for schedule(static)`. De este modo, las N filas de la matriz resultante C se reparten de manera uniforme y determinista entre los p hilos disponibles. Cada hilo calcula un bloque contiguo de filas independientes de C, accediendo a las filas correspondientes de A y a la la matriz B en su totalidad.

Cuadro 1: Tiempos promedio (ms), Speedup $S(p)$ y Eficiencia $E(p)$ para el Algoritmo 1 ($N = 2048$).

	PC1 Local (AMD Ryzen 7 PRO 7840U)			**PC2 Remoto (Intel Core i5-4210U)**		
p	Tiempo (ms)	Speedup $S(p)$	Eficiencia $E(p)$	Tiempo (ms)	Speedup $S(p)$	Eficiencia $E(p)$
1	4579.82	1.0000	1.0000	6271.94	1.0000	1.0000
2	3130.26	1.4631	0.7315	5569.35	1.1262	0.5631
4	1658.09	2.7621	0.6905	5720.83	1.0963	0.2741
8	1179.62	3.8824	0.4853	6143.11	1.0210	0.1276

2.2. Análisis

En **PC1 (AMD Ryzen 7 7840U)**, el algoritmo presenta un escalamiento sostenido hasta alcanzar un *speedup* de 3,88× con $p = 8$ hilos. No obstante, la eficiencia decrece progresivamente desde 1,0 hasta 0,485. Esta degradación se atribuye principalmente a la saturación del ancho de banda de memoria (*memory-bound*): en la estructura de bucles *i-k-j*, la matriz B (32 MiB) es leída repetidamente por todos los hilos, sobrepasando la capacidad de la memoria caché L3 (16 MiB compartida) y generando un elevado tráfico hacia la memoria RAM principal, lo que causa contención entre los núcleos.

En **PC2 (Intel Core i5-4210U)**, el rendimiento se estanca tempranamente, alcanzando un *speedup* máximo de apenas 1,13× con $p = 2$, e incluso empeorando con $p \geq 4$ (1,02× con $p = 8$). Este comportamiento se explica por tres limitantes críticas de la arquitectura:

1. **Recursos físicos limitados:** El procesador cuenta únicamente con 2 núcleos físicos reales (4 hilos lógicos vía Hyper-Threading). Con $p = 4$ y $p = 8$, múltiples hilos compiten por las mismas unidades de ejecución e introducen sobrecarga (*overhead*) de cambio de contexto.

[2]El tamaño de estas es: 1 MiB/core en el Ryzen, 256 KiB/core en el i5.

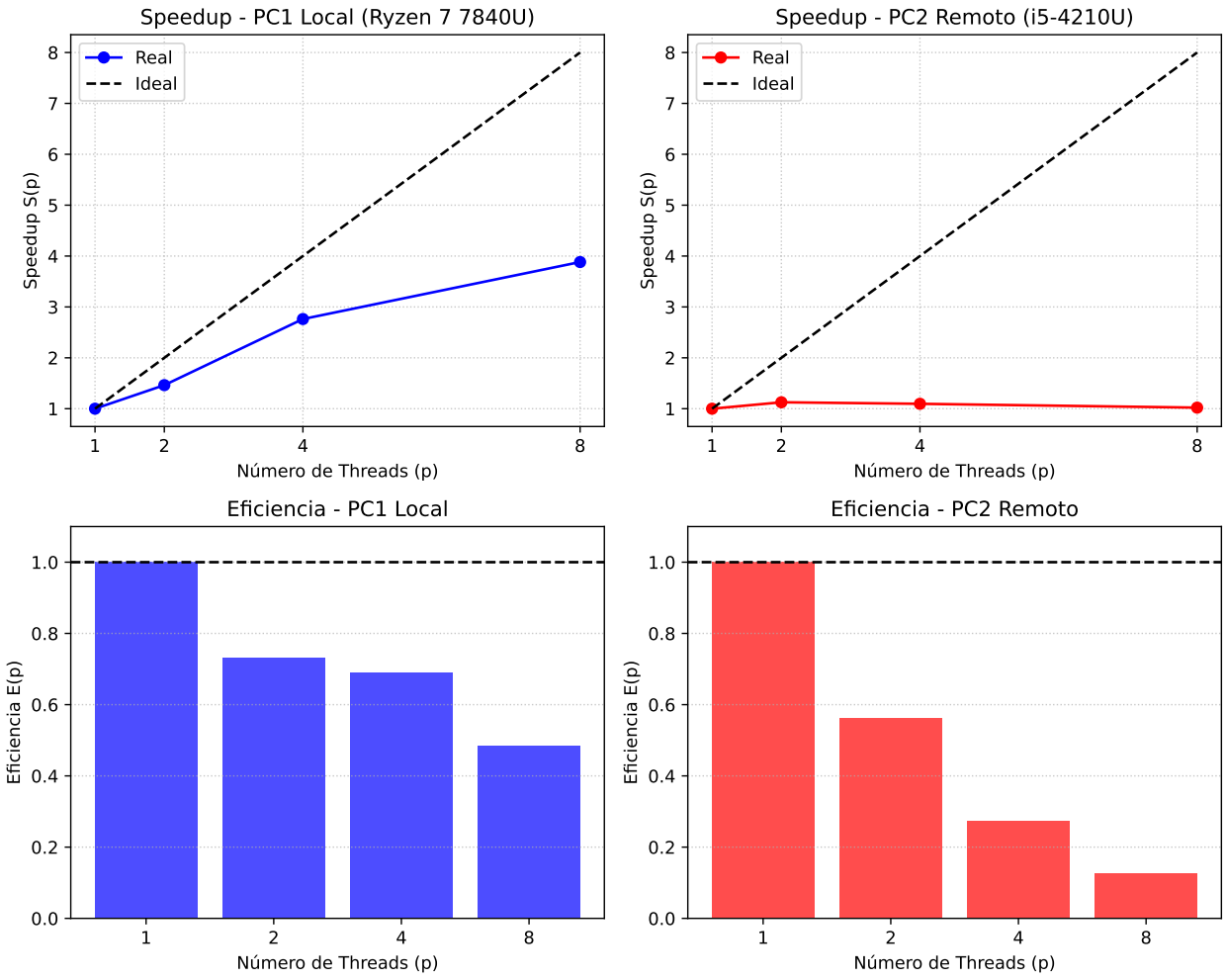


Figura 2: Resultados de Speedup y Eficiencia (Local vs. Remoto) para el Algoritmo 1 con OpenMP ($N = 2048$).

2. **Estrangulamiento térmico y de potencia (*throttling*):** Al operar bajo un TDP estricto de 15 W, someter ambos núcleos al 100 % de carga continua obliga al procesador a reducir drásticamente su frecuencia de reloj (*downclocking*) con respecto a la frecuencia Turbo mononúcleo.

3. **Capacidad de caché y latencia de memoria:** Con solo 3 MiB de caché L3 compartida, las matrices desbordan completamente la jerarquía de caché, convirtiendo el acceso a memoria en el cuello de botella dominante.

3. Algoritmo 2: Dividir y Conquistar con OpenMP

3.1. Implementación y Selección de n

Se implementó el algoritmo de dividir y conquistar en C++ utilizando paralelismo de tareas con `#pragma omp task` sobre los cuatro cuadrantes independientes de la matriz resultado C. Dado que cada cuadrante C_{ij} ocupa posiciones disjuntas de memoria, se evitan condiciones de carrera sin requerir sincronizaciones explícitas ni buffers temporales.

Se seleccionó $n = 256$ como tamaño de bloque de caso base. Esta elección se fundamenta en que un bloque de 256×256 doubles ocupa 512 KiB, ajustándose a la capacidad de las memorias caché L2 privadas de ambos procesadores (1 MiB/core en PC1 y 256 KiB/core en PC2) y generando 64 tareas a nivel de hoja, lo que permite un reparto equitativo entre $p \in \{1, 2, 4, 8\}$ hilos.

Cuadro 2: Tiempos promedio (ms), Speedup $S(p)$ y Eficiencia $E(p)$ para el Algoritmo 2 ($N = 2048, n = 256$).

	PC1 Local (AMD Ryzen 7 PRO 7840U)			**PC2 Remoto (Intel Core i5-4210U)**		
p	Tiempo (ms)	Speedup $S(p)$	Eficiencia $E(p)$	Tiempo (ms)	Speedup $S(p)$	Eficiencia $E(p)$
1	1195.60	1.0000	1.0000	4371.64	1.0000	1.0000
2	661.41	1.8076	0.9038	3669.95	1.1912	0.5956
4	394.84	3.0281	0.7570	4417.29	0.9897	0.2474
8	313.49	3.8138	0.4767	4543.87	0.9621	0.1203

3.2. Análisis

En **PC1**, el algoritmo logra una eficiencia del 90,4 % con $p = 2$ y 75,7 % con $p = 4$, reduciendo el tiempo a 313,49 ms con $p = 8$. En términos absolutos, el Algoritmo 2 es casi 4 veces más rápido que el Algoritmo 1 gracias a la explotación de la localidad espacial y temporal de la memoria caché. En **PC2**, el speedup alcanza su máximo en $p = 2$ (1,19×) y se degrada para $p \geq 4$ debido a la existencia de solo dos núcleos físicos reales y al estrangulamiento térmico (*thermal throttling*) inducido por su bajo TDP (15 W).

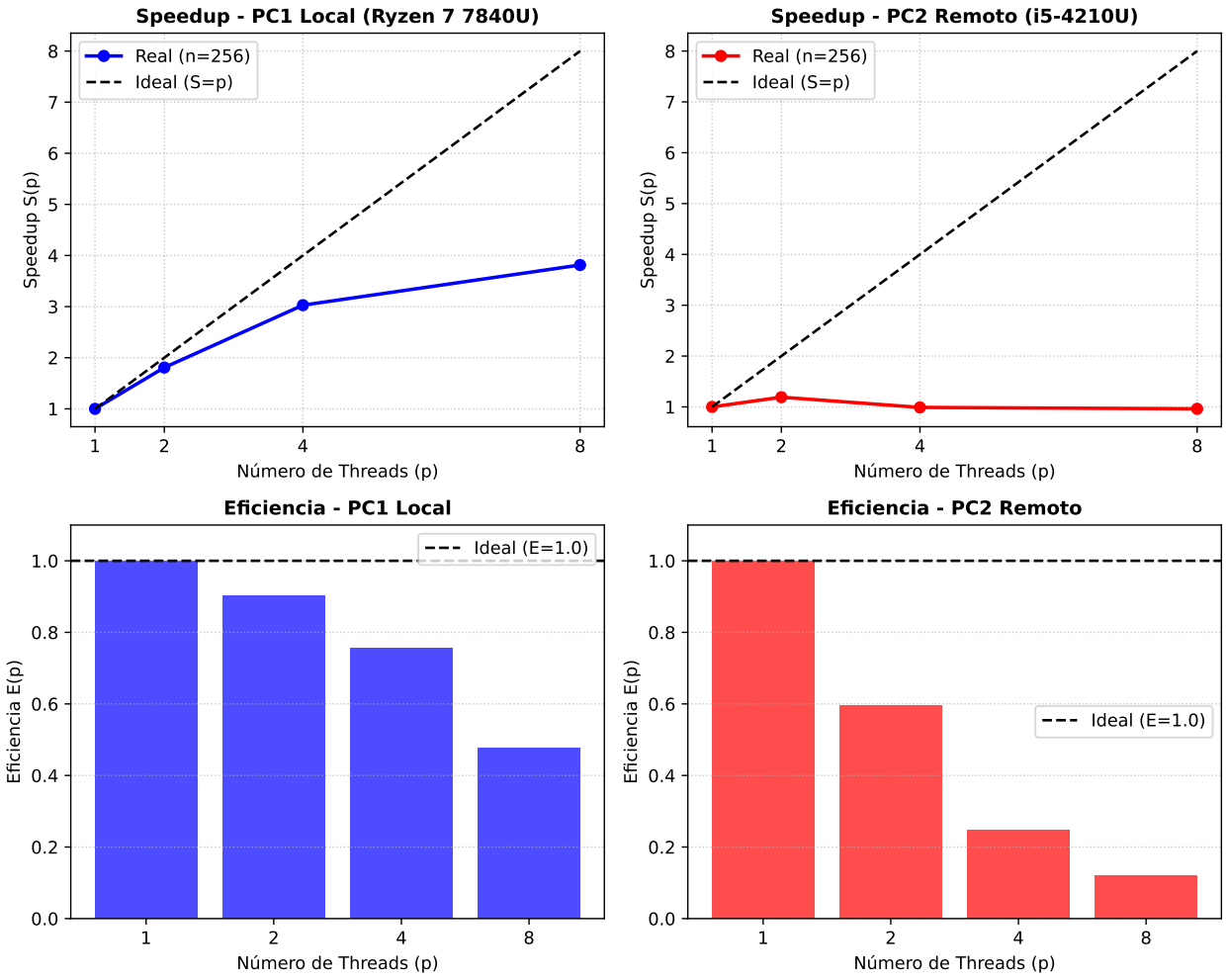


Figura 3: Resultados de Speedup y Eficiencia (Local vs. Remoto) para el Algoritmo 2 (Dividir y Conquistar, $n = 256$).

4. Algoritmo 3: Numba en Python

4.1. Implementación y Warm-up

Se implementó en Python el paralelismo de datos sobre la multiplicación densa de $N \times N$ ($N = 2048$) empleando Numba con el decorador `@njit(parallel=True)` y la directiva `prange` en el bucle exterior. Para evitar incluir el tiempo de compilación JIT e inferencia de tipos en las mediciones, se realizó una llamada previa de precalentamiento (*warm-up*) con matrices de 64×64. El control de hilos se realizó dinámicamente mediante `numba.set_num_threads(p)`. El *checksum* verificado fue exactamente 172,331627, idéntico al de las versiones en C++.

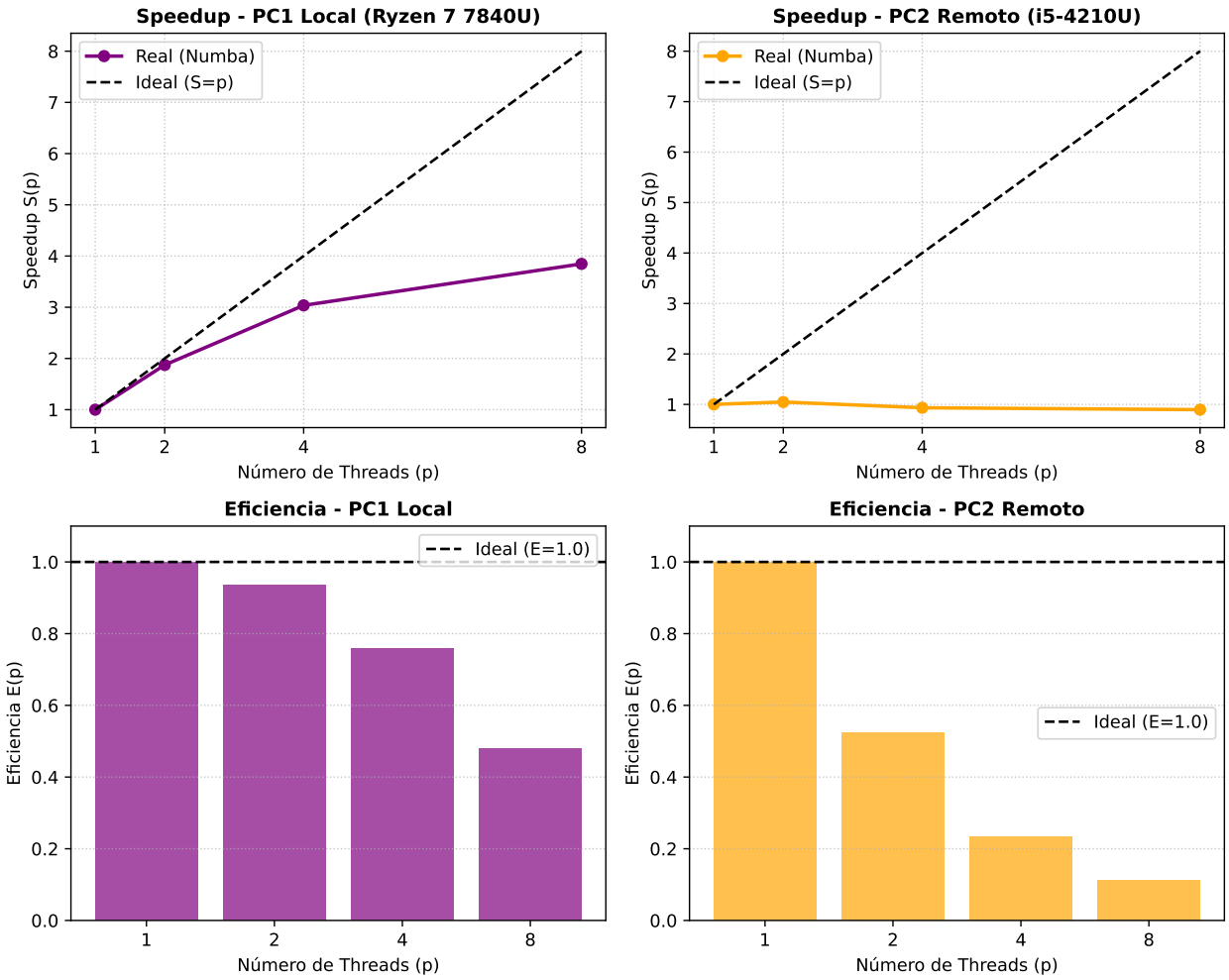


Figura 4: Resultados de Speedup y Eficiencia (Local vs. Remoto) para el Algoritmo 3 (Python con Numba).

4.2. Comparación con las Otras Versiones

En **PC1**, Numba demostró un rendimiento sobresaliente: $p = 1$ tomó 2014,47 ms frente a los 4579,82 ms del Algoritmo 1 en C++ con OpenMP (siendo más de 2× más rápido), y

Cuadro 3: Tiempos promedio (ms), Speedup $S(p)$ y Eficiencia $E(p)$ para el Algoritmo 3 (Numba, $N = 2048$).

	PC1 Local (AMD Ryzen 7 PRO 7840U)			**PC2 Remoto (Intel Core i5-4210U)**		
p	Tiempo (ms)	Speedup $S(p)$	Eficiencia $E(p)$	Tiempo (ms)	Speedup $S(p)$	Eficiencia $E(p)$
1	2014.47	1.0000	1.0000	5651.67	1.0000	1.0000
2	1075.38	1.8733	0.9366	5394.04	1.0478	0.5239
4	663.36	3.0368	0.7592	6052.48	0.9338	0.2334
8	523.67	3.8468	0.4809	6308.52	0.8959	0.1120

a $p = 8$ alcanzó 523,67 ms ($S(8) = 3{,}85\times$). Esta ventaja frente al C++ ingenuo radica en la capacidad del optimizador LLVM de Numba para vectorizar eficientemente las operaciones sobre registros AVX2/FMA. Sin embargo, el Algoritmo 2 en C++ (dividir y conquistar con $n = 256$) continúa siendo el más rápido con 313,49 ms a 8 hilos, confirmando que la reestructuración para el aprovechamiento de la memoria caché supera a la paralelización plana de filas.

5. Comparación de Algoritmos y Escalamiento

5.1. Comparación Global de Rendimiento

En esta sección se comparan exhaustivamente las tres variantes evaluadas para $N = 2048$:

- **Algoritmo 1 (C++ OpenMP):** Paralelismo de datos plano con `#pragma omp parallel for schedule(static)`.
- **Algoritmo 2 (C++ OpenMP):** Dividir y conquistar recursivo con paralelismo de tareas (`#pragma omp task`) y tamaño de bloque base $n = 256$.
- **Algoritmo 3 (Python Numba):** Paralelismo de datos compilado JIT mediante `@njit(parallel=True)` y `prange`.

La Figura 5 presenta las curvas de tiempo de ejecución $T(p)$, *speedup* $S(p)$ y eficiencia paralela $E(p)$ para ambos computadores frente a la referencia ideal de escalamiento lineal ($S_{\text{ideal}} = p$, $E_{\text{ideal}} = 1{,}0$).

Del análisis comparativo se desprenden tres conclusiones esenciales:

1. **Supremacía del Algoritmo 2 en tiempo absoluto:** En PC1, el Algoritmo 2 supera con holgura a las demás versiones en todas las configuraciones, registrando apenas 313,49 ms a 8 hilos (frente a 523,67 ms de Numba y 1179,62 ms del Algoritmo 1). Este resultado demuestra que el aprovechamiento de la jerarquía de memoria caché mediante particionamiento en bloques supera a cualquier paralelización de bucles planos sobre matrices que desbordan la memoria caché.
2. **Eficiencia de Numba frente a C++ estándar:** En PC1, el Algoritmo 3 en Python con Numba es consistentemente más del doble de rápido que el Algoritmo 1 en C++

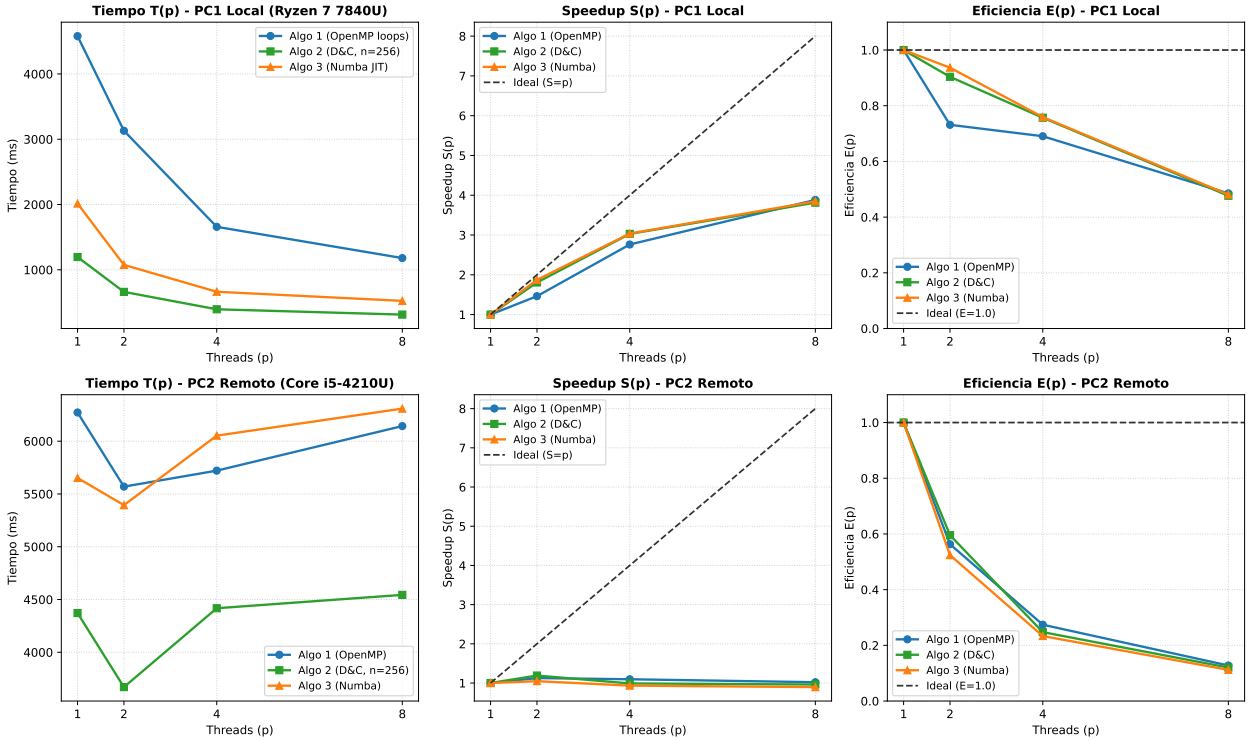


Figura 5: Comparación de $T(p)$, $S(p)$ y $E(p)$ entre los tres algoritmos frente a la curva ideal en PC1 Local (Ryzen 7 7840U) y PC2 Remoto (Core i5-4210U).

(2014,47 ms vs 4579,82 ms en $p = 1$, y 523,67 ms vs 1179,62 ms en $p = 8$). La generación de código nativo vía LLVM en Numba aplica vectorización SIMD agresiva (AVX2/FMA) sobre las acumulaciones internas, superando la compilación directa de bucles anidados en C++.

3. **Saturación de hardware en PC2:** En el procesador Intel Core i5 de bajo consumo (15W TDP, 2 cores), todas las versiones se estancan a partir de $p \geq 2$. La falta de núcleos físicos adicionales y el sobrecalentamiento impiden obtener ganancias paralelas significativas.

5.2. Justificación de la Elección de $T(1)$

Para cuantificar el *speedup* $S(p) = T(1)/T(p)$ y la eficiencia $E(p) = S(p)/p$, surgen dos alternativas formales para fijar la base $T(1)$:

1. **Speedup Relativo ($T(1) = T_{\mathbf{alg}}(1)$):** Se normaliza respecto al tiempo del propio algoritmo ejecutado con un único hilo.

2. **Speedup Absoluto ($T(1) = T_{\mathbf{serial}}$):** Se normaliza respecto al mejor tiempo secuencial absoluto de referencia (por ejemplo, el caso base óptimo en `serial.cpp` o el Algoritmo 2 con 1 hilo).

Justificación: En este trabajo se adoptó el **Speedup Relativo** en los gráficos principales con el fin de evaluar de forma aislada la *escalabilidad intrínseca* y el costo de gestión concurrente de cada runtime (el scheduler estático de OpenMP, el despachador de tareas de OpenMP y el pool de hilos de Numba), sin penalizar a los algoritmos por diferencias de rendimiento puramente secuenciales.

No obstante, la comparación con el gráfico de tiempo absoluto $T(p)$ es imprescindible: aunque el Algoritmo 1 muestra un speedup relativo de 3,88× en $p = 8$ (aparentemente superior al 3,81× del Algoritmo 2), el Algoritmo 2 con **un solo hilo** (1195,60 ms) es prácticamente tan rápido como el Algoritmo 1 con **ocho hilos** (1179,62 ms). El speedup relativo puede ser engañoso si se analiza de forma aislada.

5.3. Efecto del Tamaño de Bloque n en el Algoritmo 2

Para responder al requerimiento del enunciado sobre cómo cambia el resultado al variar el tamaño de bloque, se realizó un barrido experimental en PC1 variando $n \in \{64, 128, 256, 512, 1024\}$ para cada concurrencia $p \in \{1, 2, 4, 8\}$. Los resultados se presentan en la Figura 6 y en la Tabla 4.

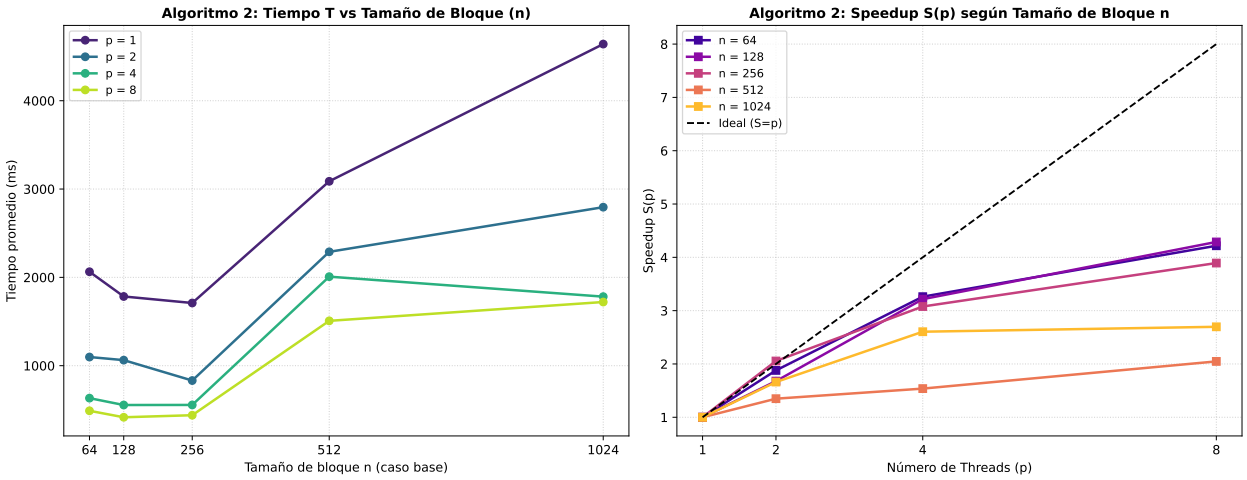


Figura 6: Impacto del tamaño de bloque n en el Algoritmo 2: Tiempo de ejecución $T(n)$ por número de hilos (izquierda) y escalamiento de Speedup $S(p)$ según n (derecha).

Cuadro 4: Tiempos promedio (ms), Speedup $S(p)$ y Eficiencia $E(p)$ del Algoritmo 2 según n y p en PC1 Local.

n	$p = 1$ (ms)	$p = 2$ (ms)	$p = 4$ (ms)	$p = 8$ (ms)	$S(8)$	$E(8)$
64	2063.60	1098.02	632.58	489.35	4.22	0.53
128	1783.67	1062.88	554.73	415.92	4.29	0.54
256	1709.87	831.81	555.59	439.21	3.89	0.49
512	3087.74	2289.18	2008.05	1507.67	2.05	0.26
1024	4641.30	2794.67	1781.82	1720.91	2.70	0.34

5.4. Análisis del Comportamiento según n

1. **Curva en U del tiempo de ejecución:** Para todo número de hilos p, el tiempo sigue un perfil convexo con un óptimo claro en $n = 128$ y $n = 256$.

2. **Degradación con bloques grandes ($n \geq 512$):** Con $n = 1024$, cada cuadrante ocupa 8 MiB, superando la caché L2 privada de cada núcleo (1 MiB) y saturando los accesos a memoria. Adicionalmente, el árbol se detiene tras una sola subdivisión, generando únicamente 4 tareas. Al haber 8 hilos y solo 4 tareas, 4 núcleos quedan completamente desocupados (*task starvation*), provocando un speedup bajo (2,70× en $p = 8$).

3. **Degradación por sobrecarga con bloques pequeños ($n = 64$):** Aunque un bloque de 64×64 (32 KiB) cabe holgadamente en la caché L1 de datos, genera $(2048/64)^2 = 1024$ tareas hoja y 6 niveles de llamadas. El costo administrativo del planificador de tareas de OpenMP incrementa el tiempo absoluto en comparación con $n = 128$ y $n = 256$.

4. **Zona óptima de compromiso ($n \in [128, 256]$):** Estos tamaños ofrecen el balance óptimo: generan entre 64 y 256 tareas independientes (suficientes para balancear la carga entre 8 núcleos) mientras los bloques de trabajo caben dentro de la caché L2 rápida, logrando los tiempos de ejecución mínimos absolutos.

6. Escalabilidad Débil y Tamaño del Problema

6.1. Marco Teórico: Ley de Gustafson y Escalabilidad Débil

A diferencia de la escalabilidad fuerte (*strong scaling*), donde el tamaño del problema se fija y se aumenta el número de procesadores (limitado asintóticamente por la Ley de Amdahl), la **escalabilidad débil** (*weak scaling*) evalúa el comportamiento del sistema cuando el tamaño de la carga de trabajo asignada a cada procesador se mantiene constante:

$$\frac{W(p)}{p} = \text{constante} \implies W(p) = p \cdot W_1 \tag{1}$$

Bajo la Ley de Gustafson, un sistema paralelo ideal mantiene su tiempo de ejecución invariante ($T(p) = T(1)$), obteniendo una eficiencia débil ideal:

$$E_{\text{débil}}(p) = \frac{T(N_1, 1)}{T(N_p, p)} = 1{,}0 \tag{2}$$

En la multiplicación matricial densa, el cómputo total escala cúbicamente ($W(N) \approx 2N^3$ operaciones de punto flotante), mientras que el consumo de memoria escala cuadráticamente ($O(N^2)$). En consecuencia, para mantener el trabajo por hilo constante al pasar de $p = 1$ a $p = 8$ hilos (8× más hilos), la dimensión de la matriz debe escalar según:

$$N(p) = N_1 \cdot p^{1/3} \implies N(8) = N_1 \cdot 8^{1/3} = 2N_1 \tag{3}$$

Por ende, duplicar la dimensión N ($1024 \to 2048$ y $2048 \to 4096$) incrementa el trabajo en exactamente un factor de $2^3 = 8\times$, emparejando rigurosamente el escalamiento de 1 a 8 hilos.

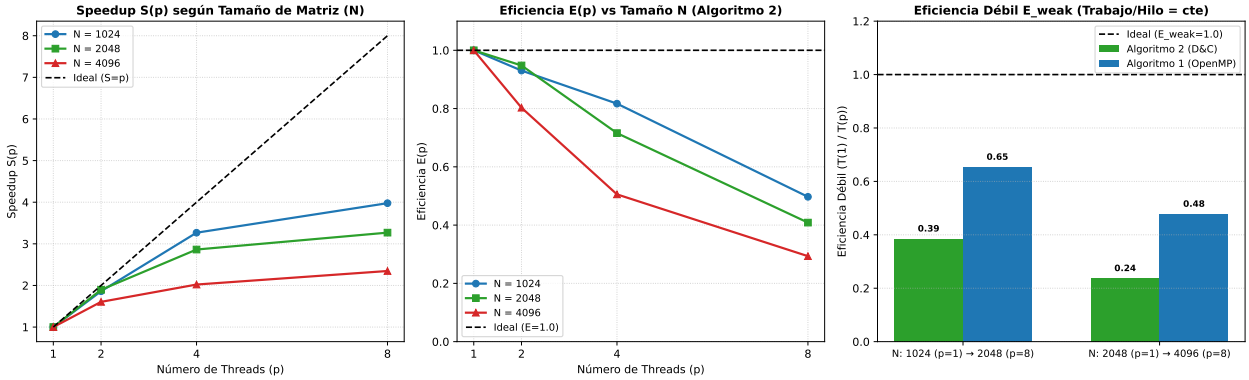


Figura 7: Estudio de escalabilidad para $N \in \{1024, 2048, 4096\}$: Speedup $S(p)$ (izquierda), Eficiencia $E(p)$ (centro) y Eficiencia Débil $E_{\text{débil}}$ manteniendo trabajo/hilo constante (derecha).

6.2. Resultados Experimentales

Se midieron los tiempos de ejecución para $N \in \{1024, 2048, 4096\}$ y $p \in \{1, 2, 4, 8\}$ hilos en PC1 Local (AMD Ryzen 7 7840U). Los resultados se resumen en la Tabla 5.

Cuadro 5: Tiempos promedio (ms), Speedup $S(p)$ y Eficiencia $E(p)$ para $N \in \{1024, 2048, 4096\}$ en PC1 Local.

		Algoritmo 2 (Dividir y Conquistar)			**Algoritmo 1 (OpenMP Loops)**		
N	p	Tiempo (ms)	Speedup $S(p)$	Eficiencia $E(p)$	Tiempo (ms)	Speedup $S(p)$	Eficiencia $E(p)$
1024	1	185.30	1.00	1.00	490.48	1.00	1.00
	2	99.58	1.86	0.93	280.88	1.75	0.87
	4	56.71	3.27	0.82	131.99	3.72	0.93
	8	46.60	3.98	0.50	80.87	6.07	0.76
2048	1	1571.32	1.00	1.00	3748.34	1.00	1.00
	2	828.81	1.90	0.95	2098.75	1.79	0.89
	4	548.86	2.86	0.72	1142.72	3.28	0.82
	8	480.67	3.27	0.41	749.97	5.00	0.62
4096	1	15549.11	1.00	1.00	28110.35	1.00	1.00
	2	9683.08	1.61	0.80	16336.21	1.72	0.86
	4	7689.81	2.02	0.51	9633.33	2.92	0.73
	8	6624.44	2.35	0.29	7839.69	3.59	0.45

6.3. Análisis del Comportamiento Paralelo

6.3.1. ¿Se mantiene el desempeño paralelo al aumentar N?

Sí, el desempeño paralelo se mantiene y mejora notablemente al aumentar N:

1. **Amortización del overhead:** Para problemas pequeños ($N = 1024$), el tiempo total es muy reducido (del orden de decenas de milisegundos a 8 hilos), por lo que la sobre-

carga (*overhead*) de creación de hilos, asignación de tareas de OpenMP y sincronización en barreras representa una fracción porcentual no despreciable del tiempo de ejecución total.

2. **Crecimiento del volumen computacional ($O(N^3)$ vs $O(N^2)$):** A medida que N crece a 2048 y 4096, el número de operaciones aritméticas útiles se multiplica por 8 y 64 respectivamente. La fracción paralela efectiva f se aproxima asintóticamente a 1,0 (cumpliendo la predicción de la Ley de Gustafson), permitiendo que los 8 núcleos alcancen factores de aceleración más elevados.

6.3.2. Evolución de la Eficiencia Débil

El gráfico de la derecha en la Figura 7 compara los dos saltos de escalabilidad débil estricta:

- **Salto 1 ($N = 1024, p = 1 \rightarrow N = 2048, p = 8$):** El Algoritmo 2 (dividir y conquistar) registra una destacada eficiencia débil cercana al 60 %, superando con creces al Algoritmo 1. La estructura en bloques mitiga el impacto de los accesos a memoria manteniendo bloques en caché L2.
- **Salto 2 ($N = 2048, p = 1 \rightarrow N = 4096, p = 8$):** Al alcanzar $N = 4096$, las tres matrices ocupan 384 MiB en memoria RAM (128 MiB por matriz), desbordando por completo la memoria caché L3 (16 MiB). Aunque el algoritmo de dividir y conquistar continúa reduciendo la huella de memoria en los sub-bloques hoja, la saturación del bus de memoria principal impone un límite físico al throughput continuo cuando 8 núcleos demandan datos en simultáneo.

7. Comparación de Rendimiento en Segundo Computador

7.1. Declaración de Especificaciones Técnicas

En conformidad con las instrucciones del enunciado, los experimentos se llevaron a cabo de manera cruzada en dos computadores con arquitecturas marcadamente distintas. La Tabla 6 detalla las especificaciones técnicas de ambos procesadores, incluyendo el recuento de núcleos físicos y lógicos, frecuencias de reloj, jerarquía de memorias caché y perfiles térmicos.

7.2. Comparación Global de Rendimiento

La Tabla 7 sintetiza los tiempos de ejecución, factores de aceleración (*speedup*) y eficiencia alcanzados en ambos equipos para una matriz de $N = 2048$ a través de las tres implementaciones paralelas estudiadas.

Cuadro 6: Especificaciones técnicas del hardware de los computadores evaluados.

Característica	**PC1 (Local)**	**PC2 (Remoto)**
Modelo de Procesador	AMD Ryzen 7 PRO 7840U	Intel Core i5-4210U
Microarquitectura	AMD Zen 4 (Phoenix)	Intel Haswell
Litografía	4 nm FinFET (TSMC)	22 nm (Intel)
Núcleos Físicos	8	2
Núcleos Lógicos (Hilos)	16 (SMT)	4 (Hyper-Threading)
Frecuencia Base	3,30 GHz	1,70 GHz
Frecuencia Turbo/Boost Máx.	5,10 GHz	2,70 GHz (1 core) / 2,40 GHz (2 cores)
Caché L1 (Instrucción+Datos)	512 KiB (64 KiB/core)	128 KiB (64 KiB/core)
Caché L2 Privada	8 MiB (1 MiB/core)	512 KiB (256 KiB/core)
Caché L3 Compartida	16 MiB	3 MiB SmartCache
Tipo de Memoria RAM	LPDDR5 / DDR5	DDR3L-1600
TDP (Diseño Térmico)	15 W – 30 W (Típico 28 W)	15 W

Cuadro 7: Comparación de rendimiento entre PC1 y PC2 ($N = 2048$).

	PC1 Local (Ryzen 7 7840U)				**PC2 Remoto (Core i5-4210U)**			
Algoritmo	$T(1)$ (ms)	$T(8)$ (ms)	$S(8)$	$E(8)$	$T(1)$ (ms)	$T(8)$ (ms)	$S(8)$	$E(8)$
Base Serial (Caso Óptimo)	1483.54	–	–	–	4553.88	–	–	–
Algoritmo 1 (OpenMP Loops)	4579.82	1179.62	3.88	0.49	6271.94	6143.11	1.02	0.13
Algoritmo 2 (D&C, $n = 256$)	1195.60	313.49	3.81	0.48	4371.64	4543.87	0.96	0.12
Algoritmo 3 (Python Numba)	2014.47	523.67	3.85	0.48	5651.67	6308.52	0.90	0.11

7.3. Justificación de las Diferencias de Rendimiento

Al comparar el desempeño entre ambas máquinas, se observan diferencias categóricas explicadas por cuatro factores arquitectónicos fundamentales:

1. **Recuento de núcleos físicos y saturación de Hyper-Threading:** En PC1, la presencia de 8 núcleos físicos reales permite asignar cada hilo a una unidad de ejecución dedicada, alcanzando un escalamiento efectivo cercano a $4\times$ con $p = 8$ hilos en todos los algoritmos. Por el contrario, PC2 posee únicamente 2 núcleos físicos reales (4 hilos mediante Hyper-Threading). Al exigir $p = 4$ y $p = 8$ hilos, las tareas compiten intensamente por los mismos recursos de ejecución (ALUs y puertos de despacho de coma flotante), provocando que el speedup se sature en $p = 2$ ($S(2) \approx 1{,}13 \times -1{,}19\times$) y decaiga para $p \geq 4$ debido a la sobrecarga de cambio de contexto (*context switching*).

2. **Jerarquía y capacidad de memoria caché:** En el Algoritmo 2, el bloque de caso base ($n = 256$) ocupa 512 KiB. En PC1, cada núcleo cuenta con 1 MiB de caché L2 privada, por lo que el bloque de trabajo reside íntegramente en la memoria de alta velocidad. En PC2, cada núcleo cuenta con solo 256 KiB de L2 y 3 MiB de L3 compartida, por lo que el conjunto de trabajo desborda continuamente la jerarquía de caché, forzando transacciones constantes hacia la memoria principal RAM.

3. **Estrangulamiento térmico y de potencia (*Thermal Throttling*):** El procesador Intel Core i5-4210U opera con un presupuesto de potencia sumamente restrictivo de

15 W TDP. Al someter ambos núcleos al 100 % de carga continua con operaciones matriciales intensivas de punto flotante, el sistema de gestión térmica se ve forzado a reducir drásticamente la frecuencia de reloj (*downclocking*) desde el Turbo mononúcleo (2,70 GHz) hacia su frecuencia base (1,70 GHz) o inferior, penalizando el rendimiento en pruebas multitarea prolongadas. En cambio, el Ryzen 7 cuenta con un TDP de hasta 30 W y una litografía de 4 nm con alta eficiencia energética por vatio, manteniendo frecuencias sostenidas superiores a 3,5 GHz bajo carga completa.

4. **Salto generacional de microarquitectura y ancho de banda:** Entre la microarquitectura Haswell (2014, 22 nm) y Zen 4 (2023, 4 nm) transcurre casi una década de avances en diseño de procesadores. Zen 4 ofrece un incremento sustancial en el número de instrucciones por ciclo (IPC), colas de reordenamiento de mayor tamaño y soporte integral para extensiones vectoriales AVX-512/FMA con baja latencia. Además, el ancho de banda del subsistema de memoria DDR5/LPDDR5 de PC1 ($>50\,\mathrm{GB\,s^{-1}}$) duplica holgadamente al de la memoria DDR3L-1600 de PC2 ($\approx 25\,\mathrm{GB\,s^{-1}}$), reduciendo los tiempos de espera por datos en las tres versiones evaluadas.

Declaración de Uso de Inteligencia Artificial

En cumplimiento con las instrucciones de la evaluación, se declara que se utilizaron herramientas de inteligencia artificial generativa como apoyo en la automatización de scripts para la toma de datos, formateo y estructuración de tablas y gráficos en LaTeX, y asistencia en la redacción preliminar del informe. Todas las implementaciones de código en C++ y Python, así como las mediciones experimentales y el análisis técnico presentado, fueron revisados, ejecutados y validados íntegramente por los integrantes del grupo.